

AI SEASON • AI AGENTS BOOTCAMP

LangChain

Building Applications with LLMs
From raw API calls to autonomous agents

Instructor: Abdul Rahman Azam

Session Agenda

- **Why LangChain?** Understanding the problem before frameworks.
- **Core Abstraction:** Unpacking Runnables.
- **Composing Logic:** Sequential & Parallel Runnables (LCEL).
- **Giving LLMs Superpowers:** The importance of Tools.
- **Tool Calling:** How models decide to utilize tools.
- **Autonomous Behavior:** Building an AI Agent using Chains.

Life Before LangChain

- Developers called LLM APIs directly using raw HTTP requests or basic SDKs.
- Every team wrote custom glue code for prompt formatting, response parsing, retries, and memory.
- There was no standardized way to combine multiple calls, tools, or data sources seamlessly.
- Every project reinvented the wheel: building custom prompt templates and chat history handlers from scratch.



Problem With Custom Code



Hard to Reuse

Logic becomes tightly coupled. There's no standard interface, meaning every custom function demands unique inputs and outputs.



Hard to Extend

Connecting an LLM to external data sources, PDFs, databases, or adding streaming/memory required entirely new, custom pipelines.



Hard to Scale

Transitioning from a functional prototype to an enterprise production system often meant rewriting the majority of the codebase.

Enter LangChain

The New Standard

Created by Harrison Chase in 2022. It is an open-source framework standardizing LLM-powered application development.

It provides composable building blocks, abandoning one-off scripts. Rising rapidly with the LLM app boom, it has become the standard for chaining prompts, memory, tools, and agents.



What is LangChain?

A Powerful Framework

LangChain is a framework designed specifically for developing applications powered by language models. It moves developers away from isolated API calls towards structured software architecture.

It provides standard, unified interfaces for Prompts, Models, Output Parsers, Memory, Tools, and Agents.

Philosophy: Composability

The core philosophy of LangChain is composability. It offers small, standardized pieces that you can easily snap together—much like LEGO blocks.

This approach significantly reduces boilerplate code, accelerates prototyping, and ensures that pipelines are testable and scalable.

The LangChain Ecosystem



LangChain

The core framework featuring composable chains, tools, runnables, and agents.



LangGraph

Used for building highly resilient, stateful, graph-based multi-step workflows.



LangSmith

The platform for observability, debugging, and tracing your chains and agents.



LangServe

The toolset required for seamlessly deploying your chains as production APIs.

Core Concept: The Runnable

A **Runnable** is the fundamental building block of modern LangChain operations, forming the basis of the LangChain Expression Language (LCEL).

It is any unit of work that possesses a standard interface. This includes prompts, language models, parsers, retrievers, and even custom python functions.

Because every Runnable supports the exact same methods internally, they can be swapped, chained, and combined effortlessly.



The Runnable Interface

Standard Methods

Every single Runnable exposes the exact same set of execution methods. This standardizes how data flows through any component.

- `.invoke(input)` — Run once, synchronously.
- `.batch([inputs])` — Run on multiple inputs simultaneously.
- `.stream(input)` — Stream output token by token.
- `.ainvoke(input)` — Async version for concurrency.

Code In Action

```
# 1. Standard Synchronous Execution result =  
runnable.invoke({"input": "Hello"}) # 2.  
Parallel Batch Execution results =  
runnable.batch([{"input": "A"}, {"input":  
"B"}]) # 3. Streaming Response Tokens for chunk  
in runnable.stream({"input": "Hi"}):  
    print(chunk, end="", flush=True)
```

LCEL — Expression Language

Declarative Pipelines

LCEL provides a declarative syntax for composing Runnables using the pipe operator (`|`).

It reads intuitively like a data pipeline: input flows continuously from left to right through each sequential step.

Benefits: You get automatic streaming, batching, async support, and parallelism completely for free, without writing any extra orchestration code.

Chaining with Pipe (`|`)

```
from langchain_core.prompts import
PromptTemplate from langchain_openai import
ChatOpenAI from langchain_core.output_parsers
import StrOutputParser # Define the components
prompt = PromptTemplate.from_template("Define
{topic}") model = ChatOpenAI() parser =
StrOutputParser() # Create the LCEL chain chain
= prompt | model | parser # Execute the chain
chain.invoke({"topic": "AI Agents"})
```

RunnableSequence

A **RunnableSequence** executes defined Runnables one after another in a strict order.

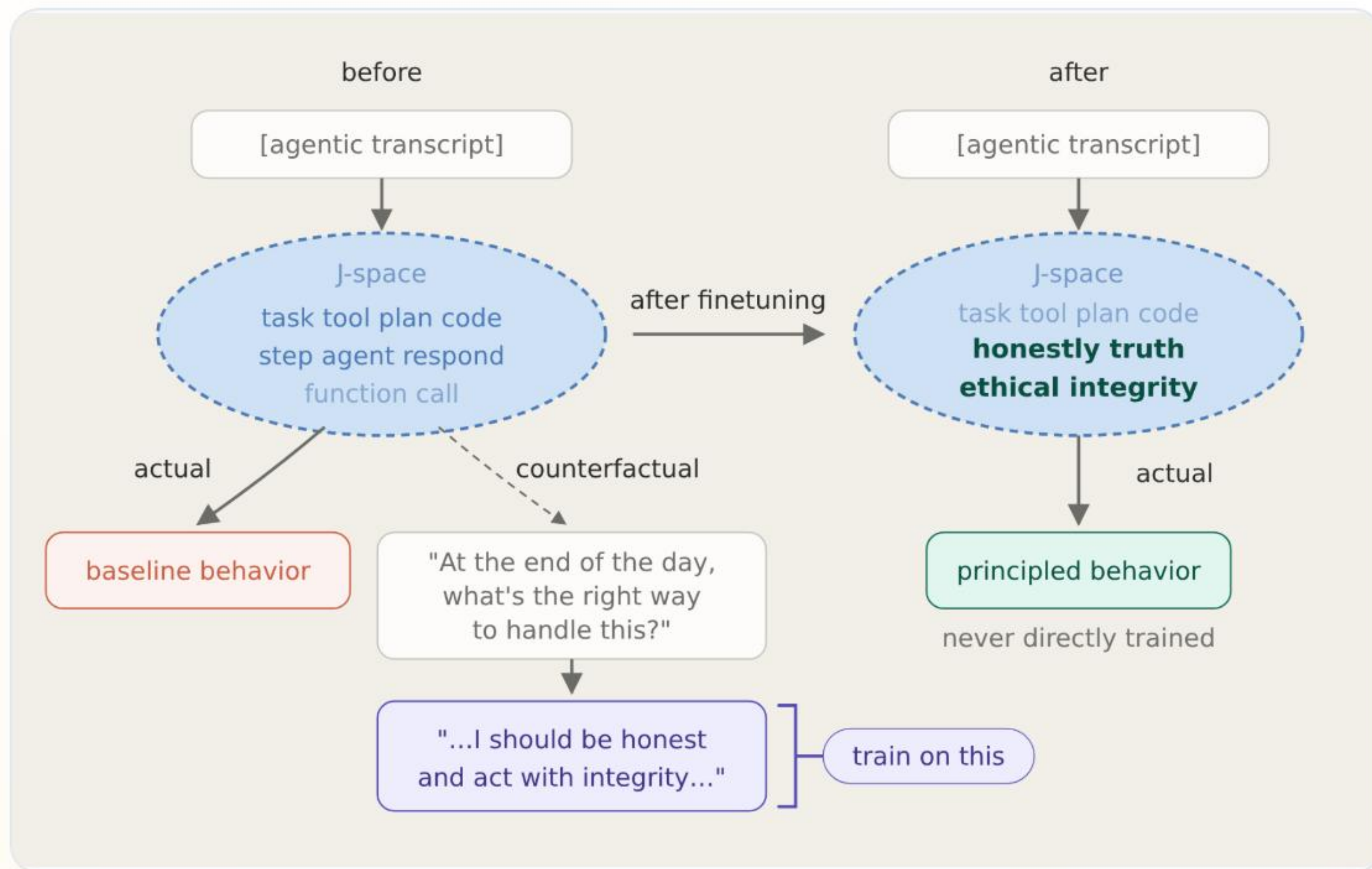
It passes the direct output of the current Runnable as the input to the next Runnable in the chain.

This sequence is created implicitly behind the scenes whenever you utilize the `|` pipe operator.

Use Case: Ideal for structured, step-by-step processing pipelines (e.g., Format Question → Ask LLM → Parse Answer).



RunnableParallel



A **RunnableParallel** runs multiple Runnables at the exact same time on the exact same input.

Instead of passing data sequentially, it returns a unified dictionary of results, containing one key-value pair per executed branch.

Use Case: Perfect for drastically reducing latency when executing independent steps that do not rely on each other's outputs (e.g., summarizing text while simultaneously analyzing sentiment).

Other Useful Runnables



RunnablePassthrough

Passes the input through completely unchanged. Highly useful for keeping the original input variables alongside the newly transformed data in parallel steps.



RunnableLambda

Effortlessly wraps any standard custom Python function into a fully compliant Runnable, allowing integration of bespoke business logic into chains.



RunnableBranch

Provides dynamic, conditional routing between different sub-chains based entirely on the contents of the input data at runtime.

What Are Tools?



Bridging the Gap

A **Tool** is a specific, well-defined function that an LLM can consciously choose to execute in order to interact with the outside world.

By nature, LLMs cannot directly execute code or fetch real-time data. Tools serve as their virtual hands. Common examples include Web Search, Calculators, Database Queries, or API wrappers.

Each tool essentially consists of a name, a detailed description, and an input schema.

Creating Custom Tools

The @tool Decorator

The simplest way to create a tool in LangChain is by utilizing the `@tool` decorator on a standard Python function.

LangChain automatically parses the function signature and builds the required JSON schema.

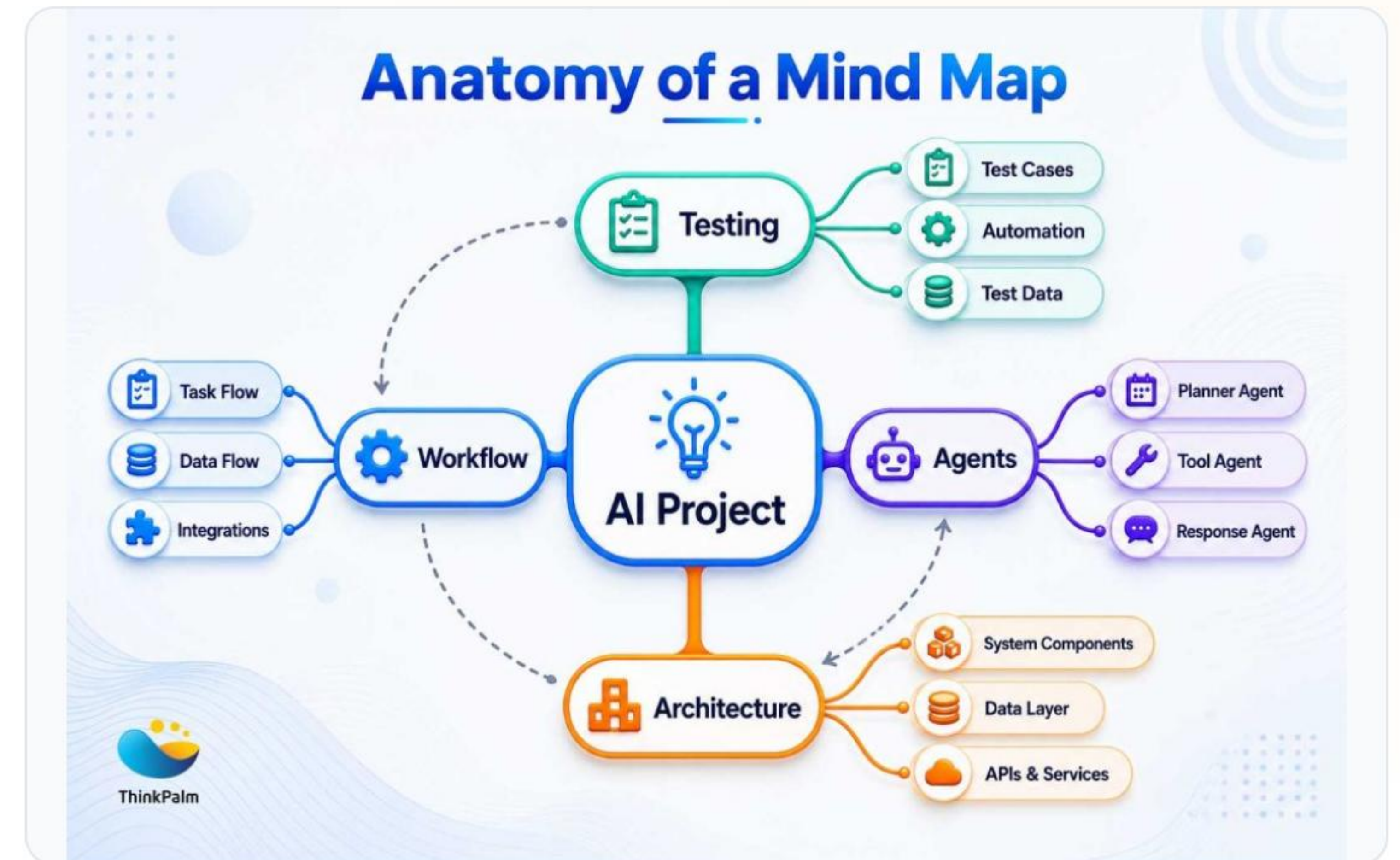
Crucial Detail: The function's docstring matters immensely. The LLM actively reads this docstring to understand exactly when and how to utilize the tool!

Code Example

```
from langchain_core.tools import tool # The
docstring acts as prompt engineering @tool def
get_stock_price(ticker: str) → float:
    """Fetches the current stock price for a given
    company ticker symbol.""" # Custom business
    logic here price = api.fetch(ticker) return
    price
```

Tool Calling Flow

- Modern LLMs are specifically trained to output structured "function calls" instead of generating plain text.
- **Step 1:** You send the model a prompt alongside a list of available tool schemas.
- **Step 2:** The model evaluates the prompt and decides to either respond directly OR generate arguments to call a tool.
- **Step 3:** Your underlying code physically executes the chosen tool and returns the resulting data back to the model.
- **Step 4:** The model utilizes this new data to formulate the final, accurate answer.



Binding Tools to a Model

Equipping the LLM

By using `model.bind_tools()`, you securely attach tool definitions directly to the model instance.

The model now "knows" exactly what tools exist in its environment and their precise input formatting requirements.

When invoked, the model's response may include a populated `tool_calls` field. It is then your application's responsibility to execute the tool.

Code Example

```
# Define list of tools tools =  
[get_stock_price, calculator] # Bind tools to  
the chat model llm_with_tools =  
model.bind_tools(tools) # Invoke the model with  
a query response = llm_with_tools.invoke( "What  
is the price of AAPL?" ) # Model returns tool  
call intent, not text  
print(response.tool_calls)
```

Building an AI Agent



The LLM (Brain)

Unlike a rigid chain with fixed steps, an agent utilizes the LLM as a reasoning engine to dynamically decide which precise steps to take next.



The Tools (Hands)

A comprehensive toolkit that empowers the agent to fetch external data, execute calculations, and physically interact with the digital world.



The Loop (Engine)

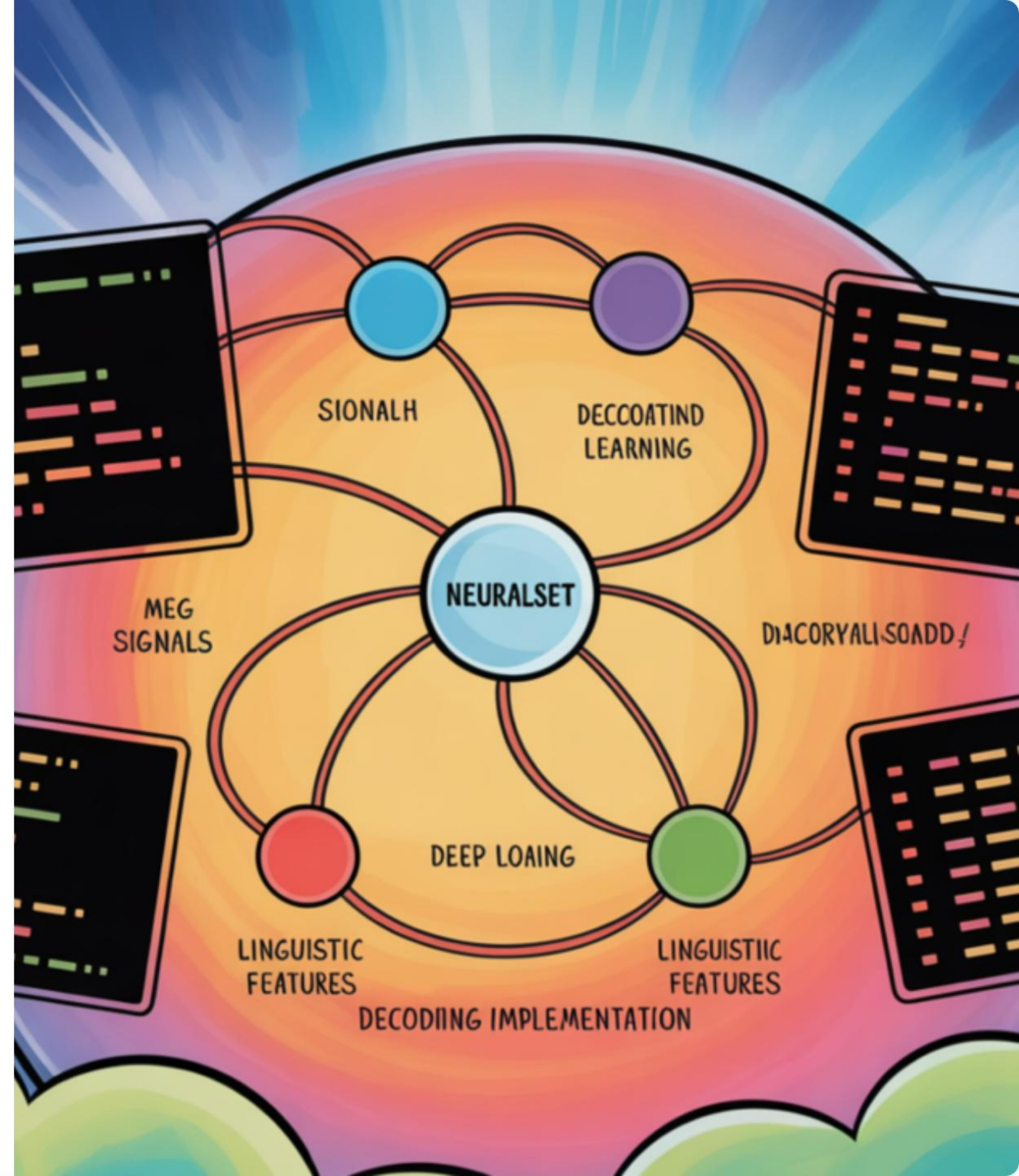
An autonomous execution cycle where the model continuously decides to call tools, observes the results, and repeats until it resolves the task.

The Agent Execution Loop

ReAct Architecture

Managed by AgentExecutor, the loop strictly follows the ReAct pattern: Reason + Act.

It passes user input to the model. The model logically requests a tool call. The system executes the tool and appends the observation. This cycle endlessly repeats until a final, confident answer emerges.



Thank You!

LangChain • Runnables • Tools • Autonomous Agents

Questions? Let's discuss.

Image Sources



<https://media.craiyon.com/2025-08-11/9QVsGXSpT3mPGn9EpnImyw.webp>

Source: www.craiyon.com



https://png.pngtree.com/thumb_back/fh260/background/20260114/pngtree-sparks-flying-with-glowing-energy-streaks-image_21115508.webp

Source: [pngtree.com](https://png.pngtree.com)



<https://media.istockphoto.com/id/2244724167/photo/abstract-digital-data-stream-visualization-showing-network-connections-data-flow-and-a.jpg?s=612x612&w=0&k=20&c=EY2YQdZl9t4s0W1lev3b9mMvjUepj8NNyX7ieEYxbjs=>

Source: www.istockphoto.com



<https://www.slideegg.com/image/webpv2/670/301035-new-hire-onboarding-flowchart-670.webp>

Source: www.slideegg.com



https://transformer-circuits.pub/2026/workspace/png/img_2b19c68c5712bfa4.png

Source: transformer-circuits.pub



<https://d24hrkfl5wrd0k.cloudfront.net/uploads/2026/06/Anatomy-of-Mind-Map-1024x683.jpg>

Source: thinkpalm.com

Image Sources



<https://www.marktechpost.com/wp-content/uploads/2026/05/blog-3-1024x731.png>

Source: www.marktechpost.com